Fast matrix multiplication algorithms in Rust

Computational laboratory project report

Alberto Defendi
Project supervisor: Leonardo Robol

Abstract

We develop a Rust crate (library) that given matrices $A, B \in \mathbb{R}^N$ computes their product $C = AB$ using fast matrix multiplication algorithms, for any given CP decomposition.

Contents

1 Introduction **1**
1.1 Notation and tensor preliminaries . . . 1

2 Fast matrix multiplication **2**
2.1 Evaluation of bilinear form using low-rank tensor decompositions . . . 2
2.2 Recursive matrix multiplication . . . 3
2.3 The classic matrix multiplication algorithm tensor formulation . . . 4
2.4 Strassen algorithm using CP . . . 5

3 Project overview **6**
3.1 Fork overview and comparison with c++ version . . . 6
3.2 Algorithm schema . . . 7
3.3 Execution platform and reproducibility . . . 8

4 Practical optimizations **8**
4.1 Dynamic Peeling . . . 9
4.2 Parallel algorithms for shared memory . . . 9
4.3 Recursive cutoff strategies . . . 10

5 Experimental results **10**
5.1 Metrics . . . 10
5.2 Measuring impact of base gemm . . . 11
5.3 Comparison of MKL dgemm FFI and Faer matmul . . . 11
5.4 Comparison of the level and size recursive cutoff strategies . . . 11
5.5 Finding the optimal cutoff for the parallel case . . . 12
5.6 Numerical comparison with Benson&Ballard Strassen . . . 12

6 Conclusion & Further work **12**

7 Work reproducibility **12**

8 Acknowledgments **17**

1 Introduction

Matrix multiplication is one of the most fundamental computations in numerical linear algebra and scientific computing. We define *fast multiplication algorithms* as those that perform asymptotically fewer floating point operations, and communicate asymptotically less data than the classical algorithm. In this work, we want to show that fast matrix multiplication algorithms are sensible to the choice of base case, and perform better than their base case for larger sizes. We also provide a library function written in Rust perform matrix multiplication with these algorithms, which supports parallel execution and different general matrix matrix multiplication *gemm* libraries, and we perform our benchmarks with Intel Math Kernel Library (MKL) `dgemm` (double precision general matrix-matrix multiplication) and faer (a modern, high performance linear algebra library for the rust programming language). According to Benson and Ballard [1], fast matrix multiplication algorithms have largely been ignored by numerical libraries due to ungrounded reasons. As they state, Intel's MKL, Amd's Core Math Library (ACML) do not provide implementation of fast algorithms. In reality, fast matrix multiplication algorithms perform fewer recursive matrix multiplications at the expense of more matrix additions, and are faster than classical ones both in theory and in practice.

Strassen's algorithm is the most well known fast algorithm, and will be at the core of our experiments, but we develop an algorithm that supports also algorithms other algorithms among those listed in Table 2. We review those algorithms and methods to construct them in Section 2. In Section 3 we explain the technical details of our implementation, and in Section 4 we explain parallel and algorithmical optimizations. Finally, we summarize experimental results in Section 5. We summarize our findings as follow: In addition, this work also serves as an evaluation of Rust as a language in scientific computing which is currently in early adoption [2].

write

1.1 Notation and tensor preliminaries

The relevant notation for our work is in Table 1. We briefly review basic tensor preliminaries, following the notation in [3]. A *tensor* is a multidimensional array, and in this work we deal only with 3-dimensional (or 3-way) tensors $\mathcal{T} \in \mathbb{R}^{M\times N\times P}$. The *k-th frontal slice* of $\mathcal{T}$ is the matrix $\mathbf{X}_k$. For $\mathbf{u} \in \mathbb{R}^M$, $\mathbf{v} \in \mathbb{R}^N$, $\mathbf{w} \in \mathbb{R}^P$, we define the rank-1 *outer product* tensor $\mathcal{T} = \mathbf{u}\bigcirc\mathbf{v}\bigcirc\mathbf{w} \in \mathbb{R}^{M\times N\times P}$ with entries $x_{ijk} = u_i v_j w_k$; such tensor has *rank* one. Addition of tensors is defined entry-wise. The rank of a tensor $\mathcal{T}$ is defined as the minimum number of rank-one tensors that generate $\mathcal{T}$ as their sum. We define the *CP* decomposition of rank R of a tensor $\mathcal{T}$ as the sum $\mathcal{T} = \sum_{r=1}^{R} u_r \bigcirc v_r \bigcirc w_r$, and we denote it by $\mathcal{T} = [\![U, V, W]\!]$, where U, V and W are matrices with R columns given by u_r, v_r, and w_r. An important operation in the context of tensors is the *mode−k product* between $\mathcal{T} \in \mathbb{R}^{N_1,N_2,N_3}$ and a matrix $U \in \mathbb{R}^{N_k\times R}$, for $k \in \{1, 2, 3\}$, which is defined as a tensor $\mathcal{Y} = \mathcal{X} \times_k U \in \mathbb{R}^{n_{k-1}\times R\times n_{k+1}}$ with $\mathbf{Y}_{(k)} = U\mathbf{X}_{(k)}$, also in components as $\mathcal{Y}(i_{k-1}, \alpha, i_{k+1}) = \sum_{i_k=1}^{3} \mathcal{X}_{i_1 i_2 i_3} U_{\alpha,i_k}$.

2 Fast matrix multiplication

2.1 Evaluation of bilinear form using low-rank tensor decompositions

Let X, Y, Z be finite-dimensional vector spaces of dimensions I, J, K respectively. A bilinear form is a mapping $f : X \times Y \to Z$ that is linear in each entry, in other words such that $f(x, \cdot)$ and $f(\cdot, y)$ are

Table 1: Summary of notation.

$\langle m, n, p \rangle$	Bilinear form representing base case matrix multiplication of an $m \times n$ matrix by a $n \times p$ matrix.
$M \times N \times P$	Dimensions of actual matrices that are multiplied ($M \times N$ matrix multiplied by $N \times P$ matrix).
$[\![\mathbf{U}, \mathbf{V}, \mathbf{W}]\!]$	Factor matrices corresponding to a tensor CP decomposition that provides a fast algorithm.
$\mathcal{T}$	Order-3 tensor.
$\mathcal{T} = \mathbf{u} \circ \mathbf{v} \circ \mathbf{w}$	Rank-1 tensor with entries $X_{ijk} = u_i v_j w_k$.
$\mathbf{T}_{(i)}$	ith frontal slice of $\mathcal{T}$, the matrix of entries $X_{:,:,i}$.
$A_{:k}$, $A_{k:}$, A_{ij}	kth column, kth row and i, j entry of matrix $\mathbf{A}$.
$\text{vec}(\mathbf{A})$	Row-order vectorization of the entries of $\mathbf{A}$.
$\text{nnz}(\cdot)$	Number of non-zero entries of an object.

linear for all $x \in X$ and $y \in Y$. Any bilinear form can be represented by a matrix M of coefficients: $f(x, y) = x^\top M y = \sum_i \sum_j m_{ij} x_i y_j$, and we can write it component-wise using a tensor $\mathcal{T}$ of coefficients t_{ijk} as follows:

$$f_k(x, y) = \sum_{i=1}^{I} \sum_{j=1}^{J} t_{ijk} x_i y_j, \qquad \text{for all } 1 \leq k \leq K, \tag{1}$$

or in more succint tensor notation,

$$f = \mathcal{T} \times_1 x^\top \times_2 y^\top. \tag{2}$$

Numerically evaluating the bilinear form in Equation 1 has an high computational cost driven by the multiplication that depends by the non-zero entries of $\mathcal{T}$. We call those multiplications *active* to emphasize their dominant computational role. We can reduce the cost in evaluating $f(x, y)$ by considering a CP decomposition $[\![U, V, W]\!]$ of rank R of $\mathcal{T}$. Denoting $\mathbf{u_l}$, $\mathbf{v_l}$, $\mathbf{w_l}$ as the columns of $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$, we have the decomposition $\mathcal{T} = \sum_{r=1}^{R} u_r \circ v_r \circ w_r$, and by performing a substitution into Equation 2 component-wise, we get

(3)

$$\begin{aligned} f_k(x, y) &= \left(\left(\sum_{r=1}^{R} u_l \circ v_r \circ w_r \right) \times_1 x^\top \times_2 y^\top \right)_k = \left(\sum_{r=1}^{R} (u_r \circ v_r \circ w_r) \times_1 x^\top \times_2 y^\top \right)_k \\ &= \sum_{i=1}^{I} \sum_{j=1}^{J} \sum_{r=1}^{R} u_{ir} v_{jr} w_{kr} x_i y_j = \sum_{r=1}^{R} \left(\sum_{i=1}^{I} u_{ir} x_i \right) \cdot \left(\sum_{j=1}^{J} v_{jr} y_j \right) w_{kr} \\ &= \sum_{r=1}^{R} (u_r^\top x) \cdot (v_r^\top y) w_{kr}, \end{aligned}$$

for every $k = 1, \ldots, K$. Since this holds for every k, we can write

$$f(x, y) = \sum_{l=1}^{r} (u_l^\top x) \cdot (v_l^\top y) w_l. \tag{4}$$

Here we highlight with $(\cdot)$ the active multiplications. Now, evaluating f requires exactly R active multiplications (and $O(r)$ additions).

2.2 Recursive matrix multiplication

Recall that matrix multiplication for two matrices $A \in \mathbb{R}^{M\times N}$ and $B \in \mathbb{R}^{N\times P}$ is defined as

$$C = AB = \left(\sum_{k=1}^{n} A_{ik}B_{kj}\right)_{\substack{1\le i\le m,\\ 1\le j\le p}} = \begin{bmatrix} A_{:1} | \cdots | A_{:n} \end{bmatrix} \begin{bmatrix} B_{1:} \\ \vdots \\ B_{n:} \end{bmatrix} = \sum_{k=1}^{n} A_{:k}B_{k:}.$$

From this expression easily follows the bilinearity of the map $(A, B) \mapsto AB$, and we indicate with $\langle M, N, P\rangle$ the form that represents the multiplication between A and B. By this property, we can rewrite Equation 2 putting $x = \mathrm{vec}(A), y = \mathrm{vec}(B)$ and $C = \mathrm{vec}(C^\top)$. This fixes the natural ordering on the entries of $\mathrm{vec}(\mathbf{A})$ and $\mathrm{vec}(\mathbf{B})$ and the inverse ordering on $\mathrm{vec}(\mathbf{C}^\top)$). This yields a representation of the tensor $\mathcal{T} \in \mathbb{R}^{MN\times NP\times MP}$ of matrix multiplication $\langle M, N, P\rangle$ from Equation 2 such that

$$\mathrm{vec}(\mathbf{C}^\top) = \mathcal{T} \times_1 \mathrm{vec}(\mathbf{A})^\top \times_2 \mathrm{vec}(\mathbf{B})^\top. \tag{5}$$

If we are given a CP decomposition of rank r of the tensor $\mathcal{T} = [\![U, V, W]\!]$, by Equation 1 this expression becomes

$$\mathrm{vec}(\mathbf{C}^\top) = \sum_{l=1}^{r} (\underbrace{\mathbf{u_l}^\top \mathrm{vec}(\mathbf{A})}_{s_l}) \cdot (\underbrace{\mathbf{v_l}^\top \mathrm{vec}(\mathbf{B})}_{t_l})\mathbf{w_l} = \sum_{l=1}^{r} (\underbrace{s_l \cdot t_l}_{m_l})w_l, \tag{6}$$

The expressions above allow us to define fast matrix multiplications algorithms for any base case $\langle m, n, p\rangle$.

2.3 The classic matrix multiplication algorithm tensor formulation

We present the $\langle 2, 2, 2\rangle$ classical matrix multiplication algorithm in the context of tensors. When $M, N, pP > 2$ we can always partition them in blocks as:

$$\begin{array}{c} \\ \lfloor M/2\rfloor \\ \lceil M/2\rceil \end{array} \overset{\overbrace{}^{\lfloor N/2\rfloor}\ \overbrace{}^{\lceil N/2\rceil}}{\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}}, \qquad \begin{array}{c} \\ \lfloor N/2\rfloor \\ \lceil N/2\rceil \end{array} \overset{\overbrace{}^{\lfloor P/2\rfloor}\ \overbrace{}^{\lceil P/2\rceil}}{\begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}}, \tag{7}$$

where the sizes of the subblocks are depicted in the above equation. This algorithmic formulation is often called *divide et impera*, as it consists in breaking the problem into smaller subproblems, and later recombining the results. In particular, we can view the product AB as the product between 2×2 block matrices

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}. \tag{8}$$

The naive algorithm proceeds by combining a set of eight matrix multiplications with four additions:

$$\begin{aligned} M_1 &= A_{11} \cdot B_{11} & M_2 &= A_{12} \cdot B_{21} & M_3 &= A_{11} \cdot B_{12} \\ M_4 &= A_{12} \cdot B_{22} & M_5 &= A_{21} \cdot B_{11} & M_6 &= A_{22} \cdot B_{21} \\ M_7 &= A_{21} \cdot B_{12} & M_8 &= A_{22} \cdot B_{22} \end{aligned}$$

$$\begin{aligned} C_{11} &= M_1 + M_2 & C_{12} &= M_3 + M_4 \\ C_{21} &= M_5 + M_6 & C_{22} &= M_7 + M_8 \end{aligned}$$

The number of flops performed by standard matrix multiplication for $N \times N$ matrices is $T(N) = 8T(\frac{N}{2}) + 4(\frac{N}{2})^2$ for $N > 1$. This can be solved by the master method to get $T(N) = 2N^3 - N^2$, where we assume that N is a power of two. In Section 4.1 we explain how to handle all dimensions.

For example, when $m = n = p = 2$, we get the tensor with frontal slices

$$\mathbf{X}_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_2 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_4 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

This yields, for example,

$$X_3 \times_1 \mathrm{vec}(A)^\top \times_2 \mathrm{vec}(B)^\top = \begin{bmatrix} a_{11} & a_{21} & a_{12} & a_{22} \end{bmatrix} \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} b_{11} \\ b_{21} \\ b_{12} \\ b_{22} \end{bmatrix} = a_{21}b_{11} + a_{22}b_{21} = c_{21}.$$

Note that the formula in Equation 6 in the context of block matrix multiplication in Equation 8 yelds to

$$\mathrm{vec}(A) = \begin{bmatrix} A_{11} \\ A_{21} \\ A_{12} \\ A_{22} \end{bmatrix}, \quad \mathrm{vec}(B) = \begin{bmatrix} B_{11} \\ B_{21} \\ B_{12} \\ B_{22} \end{bmatrix}, \quad \mathrm{vec}(C^\top) = \begin{bmatrix} C_{11} \\ C_{12} \\ C_{21} \\ C_{22} \end{bmatrix}.$$

Also the factors s_l and t_l in Equation 6 become

$$s_l = u_l^\top \mathrm{vec}(A) = \sum_{i=1}^{4} U_{li} \mathrm{vec}(A)_i, \quad t_l = V_l^\top \mathrm{vec}(B) = \sum_{i=1}^{4} V_{li} \mathrm{vec}(B)_i.$$

2.4 Strassen algorithm using CP

The most well known fast algorithm is due to Strassen, and follows the same $\langle 2, 2, 2 \rangle$ block structure:

$$
\begin{aligned}
S_1 &= A_{11} + A_{22} & S_2 &= A_{21} + A_{22} & S_3 &= A_{11} \\
S_4 &= A_{22} & S_5 &= A_{11} + A_{12} & S_6 &= A_{21} - A_{11} \\
S_7 &= A_{12} - A_{22} & & & & \\
T_1 &= B_{11} + B_{22} & T_2 &= B_{11} & T_3 &= B_{12} - B_{22} \\
T_4 &= B_{21} - B_{11} & T_5 &= B_{22} & T_6 &= B_{11} + B_{12} \\
T_7 &= B_{21} + B_{22} & & & &
\end{aligned}
$$

$$
\begin{aligned}
M_r &= S_r T_r, \quad 1 \leq r \leq 7 & & \\
C_{11} &= M_1 + M_4 - M_5 + M_7 & C_{12} &= M_3 + M_5 \\
C_{21} &= M_2 + M_4 & C_{22} &= M_1 - M_2 + M_3 + M_6
\end{aligned}
$$

This algorithm uses 7 matrix multiplications and 18 matrix additions. The number of flops performed by the algorithm is $T(N) = 7T(\frac{N}{2}) + 18(\frac{N}{2})^2$ for $N > 1$, solving the recursion the complexity is $T(N) = 7N^{\log_2 7} - 6N^2 = O(N^{\log_2 7}) = O(N^{2.81})$. This algorithm corresponds to a low-rank tensor decomposition represented by the following triplet of matrices, each with with 7 columns:

$$
U = \begin{bmatrix}
1 & 0 & 1 & 0 & 1 & -1 & 0 \\
0 & 0 & 0 & 0 & 1 & 0 & 1 \\
0 & 1 & 0 & 0 & 0 & 1 & 0 \\
1 & 1 & 0 & 1 & 0 & 0 & -1
\end{bmatrix},
$$

$$
V = \begin{bmatrix}
1 & 1 & 0 & -1 & 0 & 1 & 0 \\
0 & 0 & 1 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 1 \\
1 & 0 & -1 & 0 & 1 & 0 & 1
\end{bmatrix},
$$

$$
W = \begin{bmatrix}
1 & 0 & 0 & 1 & -1 & 0 & 1 \\
0 & 1 & 0 & 1 & 0 & 0 & 0 \\
0 & 0 & 1 & 0 & 1 & 0 & 0 \\
1 & -1 & 1 & 1 & 0 & 0 & 0
\end{bmatrix}.
$$

complete

While we can't use less than 7 multiplications for the $\langle 2, 2, 2 \rangle$ algorithm [4], there are natural extensions to Strassen algorithm:

1. We can reduce the number of addition from 18 down to 15, which leads to Strassen-Winograd algorithm .

cite

2. We can change constant on the leading term by chosing a bigger base case dimension (and using the standard algorithm (also called *gemm*) on the base case).
3. We can use blocking schemes different from $\langle 2, 2, 2 \rangle$.

We implement this extension in practice using the Rust programming language, targeting exact algorithms arising from the CP decomposition matrices made freely available by Benson&Ballard [1]. Table 2 shows a summary of fast algorithms for different base cases.

Table 2: Summary of fast algorithms. Algorithms without citation were found by the authors of [1]. The number of multiplications is equal to the rank R of the corresponding tensor decomposition. The multiplication speedup per recursive step is the expected speedup if matrix additions were free. Note that this speedup does not determine the fastest algorithm because the maximum number of recursive steps depend on the size of the subproblems created by the algorithm.

Algorithm base case	Number of multiplies (fast)	Number of multiplies (classical)	Multiplication speedup per recursive step
$\langle 2,2,3\rangle$	11	12	9%
$\langle 2,2,5\rangle$	18	20	11%
$\langle 2,2,2\rangle$ [5]	7	8	14%
$\langle 2,2,4\rangle$	14	16	14%
$\langle 3,3,3\rangle$	23	26	17%
$\langle 2,3,3\rangle$	15	18	20%
$\langle 2,3,4\rangle$	20	24	20%
$\langle 2,4,4\rangle$	26	32	23%
$\langle 3,3,4\rangle$	29	36	24%
$\langle 3,4,4\rangle$	38	48	26%
$\langle 3,3,6\rangle$ [6]	40	54	35%

3 Project overview

3.1 Fork overview and comparison with c++ version

The implementation consists in a Rust porting of the c++ project made by Benson and Ballard summarized in [1], available in the relative repository under BSD 2-Clause. We will refere to this project as "the C++ project" or "Benson&Ballard's project". They provide a fast and efficient framework for benchmarking parallel and sequential fast matrix multiplication algorithms, achieved trough code generation, using lapack and MKL dgemm for numerical computation, and OpenMP for parallelization. We develop a Rust project that aims to replicate a parallel framework for fast multiplication algorithms, but with structural differences and current limitations. The main similarities are that both use MKL for base, but Rust project also uses Faer. In the C++ project, MKL is imported, while in Rust it is binded via FFI. Besides tool choices, the C++ algorithms are code-generated by a script that parses CP tensor decomposition slices and translates the matrix multiplication expression into C++ code, while Rust loads the tensor at runtime [1] and then applies the pseudocode in Algorithm 1. The advantage of the first is speed, at the cost of more complex code and parallelism. The advantage of the second is simplicity of code, at the cost of speed (at the current state, but which it could be improved by adding optimization, we will address these issues in Section 6). The C++ project can handle multiplications between rectangular matrices, as well as APA algorithms, while our doesn't. Also, while it could work, we didn't tested subexpression eliminated algorithms.

[1] when program is launched, before doing the multiplication, which does not affect speed since the matrices decomposition is cached at runtime

3.2 Algorithm schema

We now discuss the algorithm schema for Strassen algorithm for clarity, though results generalize easily for any $\langle m, n, p \rangle$ base case. Algorithm 1 shows our implementation of Strassen's algorithm, which easily extends to other CP algorithms by changing base conditions and block sizes—this is how it is implemented in practice.

Algorithm 1 Strassen algorithm using CP factorization.

procedure STRASSEN(A, B, N)
  if STOP-RECURSING is True **then**
    return $A \cdot B$
  end if
  if $N = 2$ **then**
    STRASSEN-BASE-MATMUL(A, B)
  end if
  if $N \mod 2 > 0$ **then**
    DYNAMIC-PEELING(A, B, N) ▷ Recursive call on peeled blocks.
  end if
  Compute $\text{vec}(A)$ and $\text{vec}(B)$ with block size $\frac{N}{2}$.
  for $r = 1, \ldots, R$ **do**
    $S_r \leftarrow \sum_{i=1}^{4} U_{i,r} \, \text{vec}(A)_i$
    $T_r \leftarrow \sum_{i=1}^{4} V_{i,r} \, \text{vec}(B)_i$
    $M_r \leftarrow$ STRASSEN($S_r, T_r, \frac{N}{2}$) ▷ Recursive call.
  end for
  $\text{vec}(\mathbf{C}^\top) = \sum_{r=1}^{r} M_r w_r$
  return C.
end procedure

3.3 Execution platform and reproducibility

Platform

Since the comparison targets both sequential and parallel matrix multiplication kernels, we ran all experiments on one node of a dual-socket AMD EPYC 7763 server (64 physical cores per socket, 2 threads per core) with 2 TB of RAM. The system ran Ubuntu 24.04.5 LTS with Linux 5.15.0-179-generic on x86_64. For sequential benchmarks, library-level threading was restricted with `OMP_NUM_THREADS=1` and `MKL_NUM_THREADS=1`. These choices remove scheduler migration and library-level parallelism from the measurements to isolate microarchitectural performance and compiler optimizations without the interference of thread scheduling and synchronization overheads.

Environment

The software environment is configured and reproduced using Environment Modules. The configuration loads `gcc/13.2.0` as the base compiler toolchain, alongside the `intel-oneapi-compilers` version `2025.0.4-gcc-11.4.0`. To provide an optimized BLAS/LAPACK backend, the environment loads `intel-oneapi-mkl/2024.2.2-intel-oneapi-mpi-2021.14.1-oneapi-2025.0.4`. We are aware that this makes the

environment not consistent, but it was not possible match the correct versions without incurring into compilation errors. Ask robol

Compilation

When building the C wrappers and Rust bindings, all compilation targets are explicitly optimized for the server target architecture. In the C++ project, to support both sequential and parallel modes within a single executable, the MKL backend was dynamically linked against the GNU OpenMP threading layer (`mkl_gnu_thread` and `-lgomp`) instead of the sequential MKL library. By mistake, the C++ project was linked against LVM OpenMP passing `-liomp5`. We build the Rust code with the nightly-2026-06-23 toolchain because this release uses LLVM 22, which generates optimized vector instructions leveraging AVX2 and FMA. Since the C/C++ wrappers and Rust code compile with matching CPU targets, a custom `build.rs` script compiles the Intel MKL FFI C wrapper (`mkl.c`) using `gcc`. During linking, `build.rs` ensures that the Intel MKL dynamic libraries are correctly loaded. To eliminate the need for manual `LD_LIBRARY_PATH` configuration at runtime, the MKL library path is compiled into the executable binaries as a runtime.

4 Practical optimizations

Fast matrix multiplication algorithm such as Strassen face two significant problems in practical implementation, which are recursion cutoff and input matrices sizes not matching with base case. In the parallel case, there also scheduling problems to consider to avoid algorithm overhead.

4.1 Dynamic Peeling

In Strassen algorithm we want to multiply $\langle m, n, p \rangle$ matrices by splitting in four blocks as in Equation 7. If m, n and p are even, then it is not possible to split blocks in even sizes. The easiest fix for this is padding the matrix with zeros until an even size is reached, but this adds a significant memory waste. A more efficient approach, called *dynamic peeling*, consists in stripping the extra row off and/or column and adding their contribution to the final results. In detail,

For the matrix multiplication $C = AB$, where A is an $M \times N$ matrix and B is an $N \times P$ matrix. The matrix A is split into an $(M-1) \times (N-1)$ core matrix A_{11}, alongside its peeled boundary vectors a_{12}, a_{21}, and the scalar corner element a_{22}. Similarly, the matrix B is split into an $(N-1) \times (P-1)$ core matrix B_{11}, with its corresponding peeled components b_{12}, b_{21}, and b_{22}:

$$A = \left[\begin{array}{c|c} \overbrace{A_{11}}^{n-1} & \overbrace{a_{12}}^{1} \\ \hline a_{21} & a_{22} \end{array}\right] \begin{matrix} \} m-1 \\ \} 1 \end{matrix}, \qquad B = \left[\begin{array}{c|c} \overbrace{B_{11}}^{p-1} & \overbrace{b_{12}}^{1} \\ \hline b_{21} & b_{22} \end{array}\right] \begin{matrix} \} n-1 \\ \} 1 \end{matrix}$$

The final block matrix product is computed directly through a combination of the core Strassen multi-

plication ($A_{11}B_{11}$) and low-rank vector/scalar fixup modifications:

$$\left[\begin{array}{c|c} \overbrace{C_{11}}^{p-1} & \overbrace{c_{12}}^{1} \\ \hline c_{21} & c_{22} \end{array}\right] \begin{array}{l} \} m-1 \\ \} 1 \end{array} = \left[\begin{array}{c|c} A_{11}B_{11} + a_{12}b_{21} & A_{11}b_{12} + a_{12}b_{22} \\ \hline a_{21}B_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{array}\right]$$

Here, only the sub-product $A_{11}B_{11}$ is executed recursively using Strassen's matrix multiplication, while all other terms are computed via standard matrix multiplication.

4.2 Parallel algorithms for shared memory

Matrix multiplication is a problem subject to both computational and memory bound. The first is limited by the amount of computation the device can handle (i.e cores,cpu/gpu speed), while the latter by physical storage of the matrices (i.e available RAM memory). In a parallel implementation, when dealing with recursive algorithms which use divide-et-impera (like Strassen), it is important in practice to balance algorithmic load between threads to avoid overhead caused by both taking recursive branches and doing parallel matrix multiplication. To address this issue, we adopt the scheduling schema suggested by Benson and Ballard in their work [1]. We use Rust's Rayon to handle parallelization, and we can summarize the parallel scheduling as following:

- **BFS:** Each recursive matrix multiplication routine and the associated matrix additions are launched by Rayon as an iterable, and then are collected in an array of matrices $[M_1, \ldots, M_r]$.
- **DFS:** Each vendor matrix multiplication call uses all thread, and matrix multiplications are always fully parallelized.
- **Hybrid:** The Hybrid approach developed in [1] compensates the for the load imbalance in BFS by applying DFS on a subset of the base case problems. With L levels of recursion and P threads, the hybrid algorithm applies task parallelism (BFS) to the first $R^L - (R^L \mod P)$ multiplication. Because the number of subproblems is a multiple of P, this part is load balanced. On the remaining $R^L \mod P$ subproblems, all threads are used on each matrix multiplication (DFS).

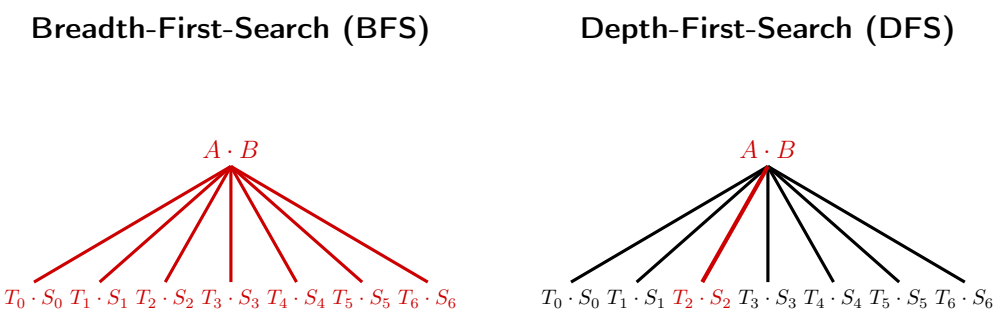


Figure 1: BFS and DFS load balancing. The parallelized tasks are in red.

4.3 Recursive cutoff strategies

In divide et impera algorithms such as Strassen, we can't afford taking many recursive steps, as spawning many children constitutes a significant recursive overhead, especially for larger matrices. Thus, finding an optimal cutoff is important for stopping the recursion and switch to standard gemm. We try two techniques: the first is breaking by size, that is when the sizes of matrices in the recursion fall under a certain size, the latter is taking a fixed number L of recursions.

The authors in [1] explain an euristical process for finding the optimal cutoff size, which can be resumed in looking at the performance plot of the gemm function, where (figure) the function exibits a ramp-up and then flatten for large dimensions. They take a recursion step only if the recursive sub-problem fall in the flat part of the curve. This is done manually by the authors. During the experimentation we developed a function to plot this curve, and its derivative (using splines) to automatically find this size. In practice, we never use this as we believe that our code is not efficient enough to implement this strategy.

5 Experimental results

5.1 Metrics

All benchmarks are recorded using Rust's system clock, performing warm-up before doing measuring each matrix multiplication. In order to compare the performance of matrix multiplication algorithms with different ocmputational costs, we use the *effective* GFLOPS metric for $\langle m, n, r \rangle$ matrix multiplication:

$$\text{effective GFLOPS} = \frac{2mnp - mp}{\text{time in seconds} \cdot 10^9}, \tag{9}$$

where the numerator can be rewritten as $2mnp - mp = mnp + m(n-1)p$, which is the number of multiplications and additions performed for classical matrix multiplication. This allows us to compare all of the algorithms on an inverse-time scale, normalized by problem size [1]. For fast algorithms, considering both multiplications and additions in the metric is more accurate than taking multiplications only [7].

5.2 Measuring impact of base gemm

We aim to understand how the choice of the base matrix multiplication impacts performance of fast algorithms. In our experiments, we use random square matrices with $64bit$ floating point entries in $(-1, 1)$ of sizes $N \in \{2, 4, \ldots, 2^k\}$. As the standard matrix multiplication algorithm implemented in gemm costs $O(n^3)$ while Strassen costs $O(n^{2.81})$, we expect it to beat gemm [2] for a large N, that is, we are interested in finding the N such that using a fast matrix algorithm is faster than doing standard matrix multiplication. Such N cannot be determined a priori, as it is dependent by factors such as platform and implementation. In other words, we expect fast algorithms to surpass the curves in Figure 2.

5.3 Comparison of MKL dgemm FFI and Faer matmul

In the project we have two different matrix multiplication routines: MKL dgemm and faer base matmul.

[2] which is in practice more optimized and a linear algorithm

- **Faer matmul:** The method is entirely written in Rust and is part of the faer library.
- **MKL dgemm:** is a routine in the Intel Math Kernel Library (MKL) for multiplying double precision matrices. In Rust, this routine runs via a Foreign Function Interface (FFI) calling the C wrapper `cblas_dgemm`. In the C++ project, the function is called directly. Both projects use the same library version on the architecture.

From Figure 2, we see that all functions have a ramp-up phase and then flatten for larger sizes—even if the parallel case is not flattening yet, we expect it to flatten after few more runs. Also, dgemm performs better than faer for larger sizes, and we believe this is due the many low-level optimizations in dgemm. Further, there is a discrepancy between Rust FFI dgemm and the C++ code which tends to decrease for larger size. We think that this might be due the type conversion which is performed during the binding.

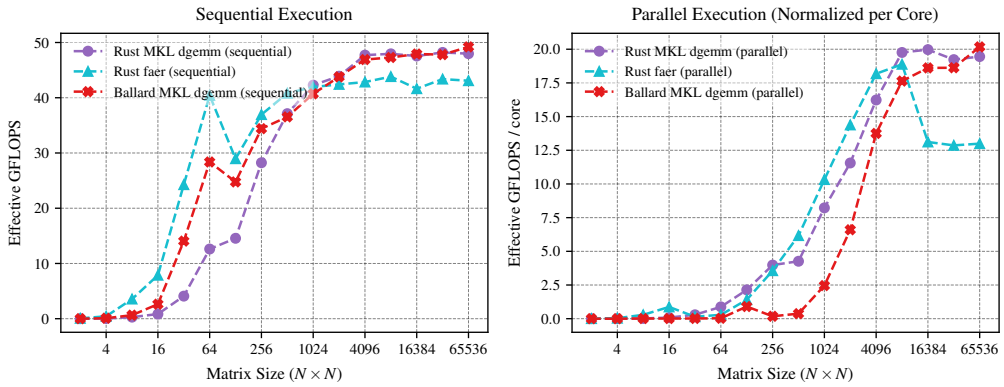


Figure 2: Comparison between MKL and faer in sequential and parallel execution.

5.4 Comparison of the level and size recursive cutoff strategies

The experimental results for sequential and parallel cutoff strategies are shown in Figures 3a and 3b, respectively. Figures 4a and 4b show comparison of Strassen and base gemm.

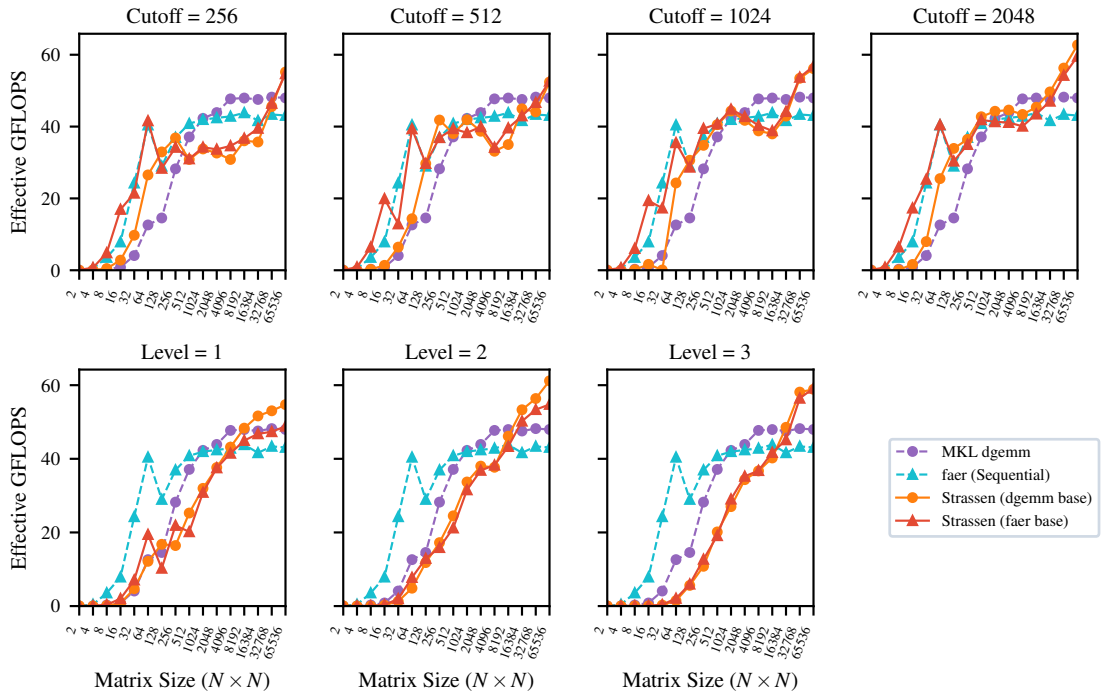
Figure 3a shows the performance of sequential Strassen variants across different recursion levels (1, 2, and 3) and cutoff sizes (256, 512, 1024, and 2048). As expected, sequential execution has lower overhead, and selecting a larger size cutoff stops recursion earlier, preventing recursive overhead.

For parallel execution, Figure 3b shows the performance (normalized per core) across larger cutoff sizes (2048, 4096, 8192, and 16384) in the first row, and recursion levels (1, 2, and 3) in the second row. Because spawning tasks in parallel introduces scheduling overhead, a larger cutoff size is needed in practice to maximize parallel efficiency, as smaller subproblems do not benefit from thread-level parallelism.

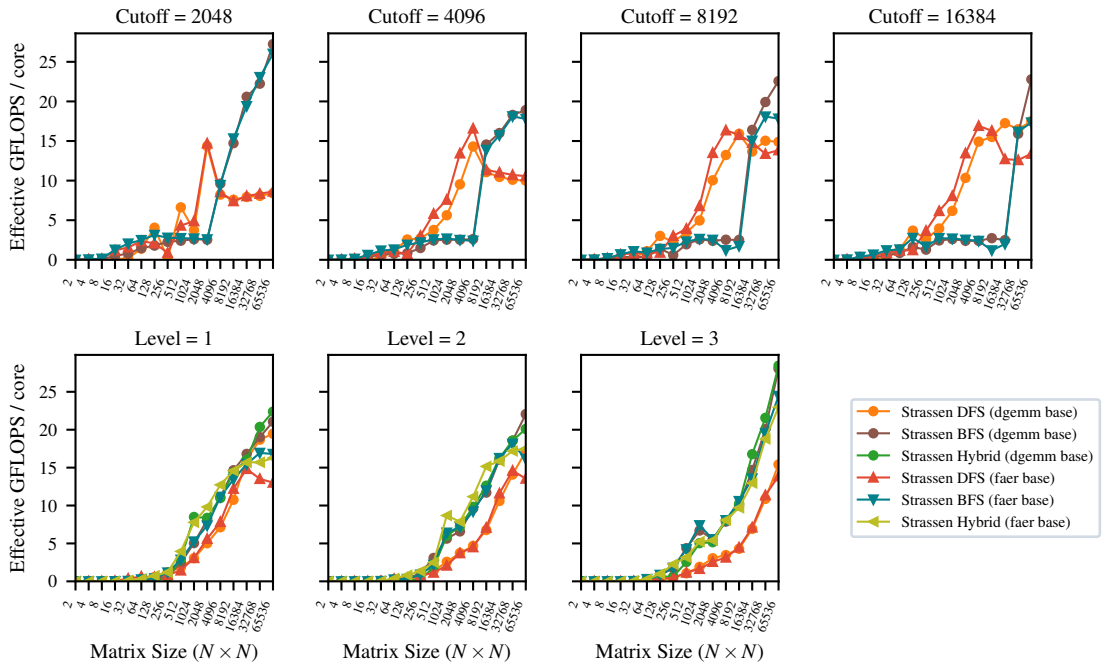
5.5 Finding the optimal cutoff for the parallel case

Figure 5 shows different recursive size thresholds for the parallel case. Compared to the sequential case in Figure 3a, optimal performance is found for larger cutoff size, such as greater equal to 8102. This is due the fact that smaller cases trigger more recursive steps and cause thread oversubscription.

Discuss variants dgemm and faer

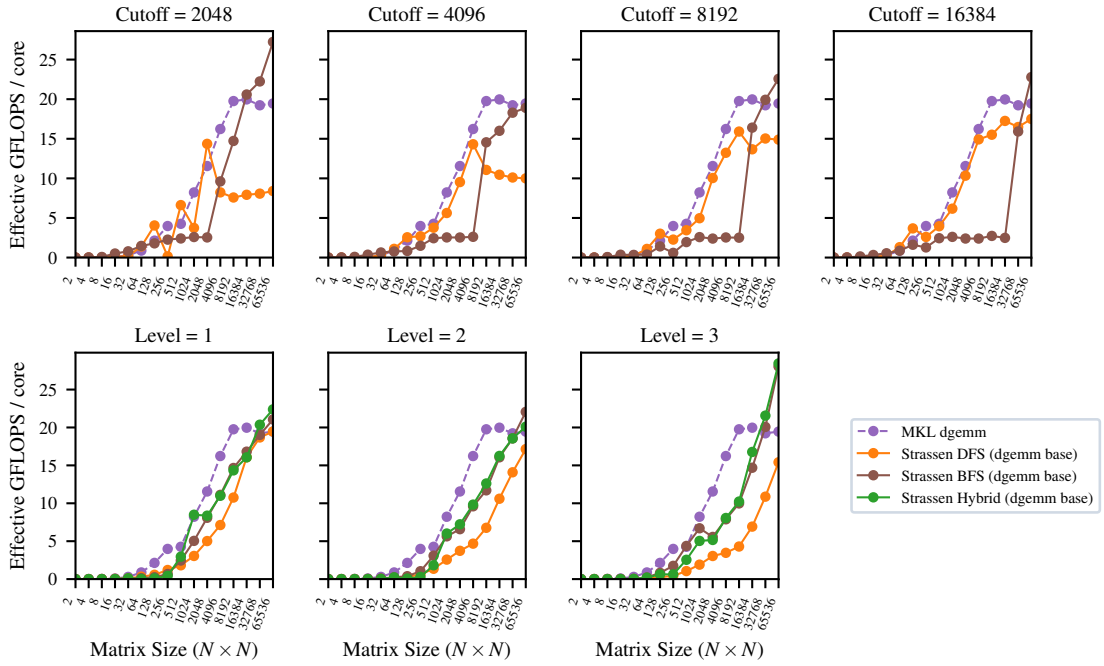


(a) Sequential execution.

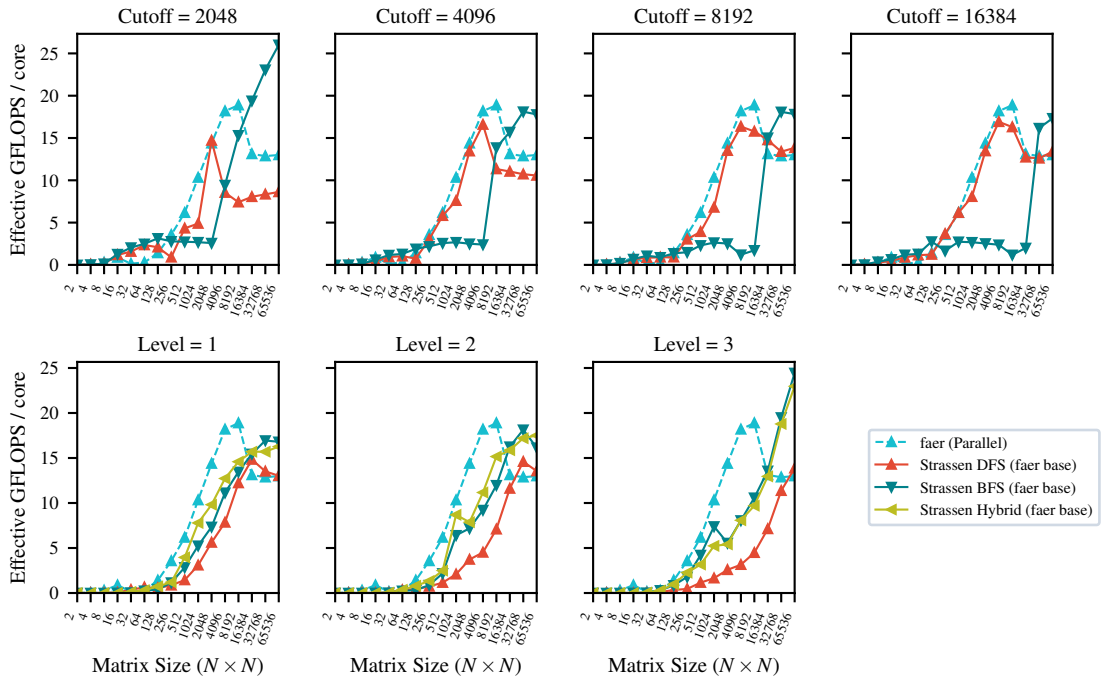


(b) Parallel execution (normalized per core).

Figure 3: Performance comparison of Strassen variants (DFS, BFS, Hybrid) across recursion levels and size cutoffs in sequential and parallel executions.

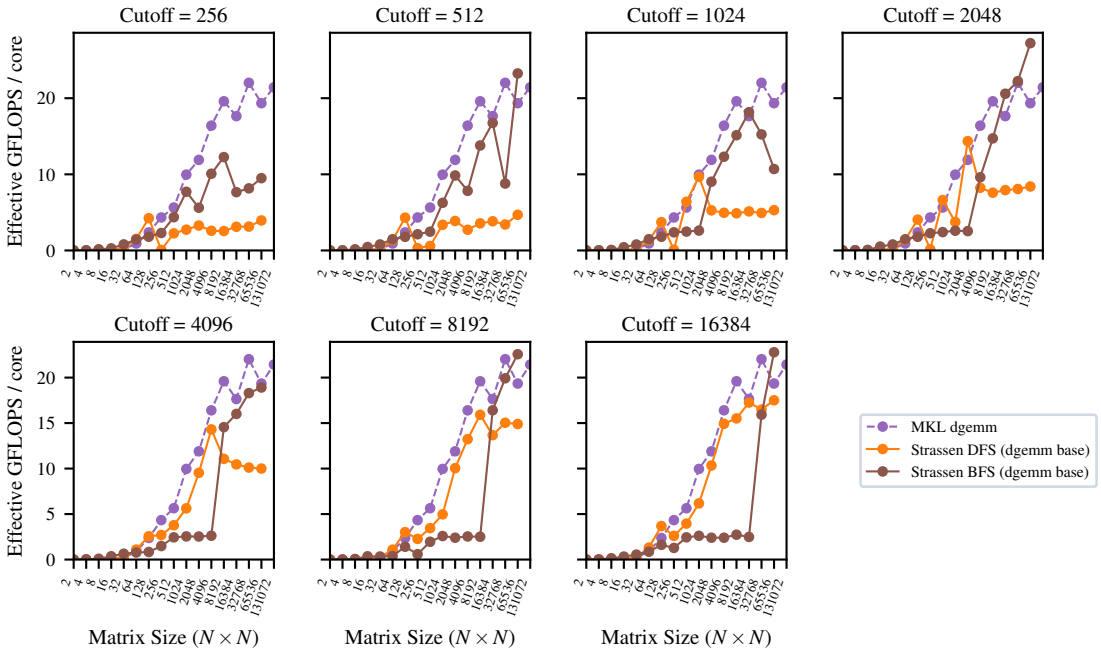


(a) Parallel dgemm base.

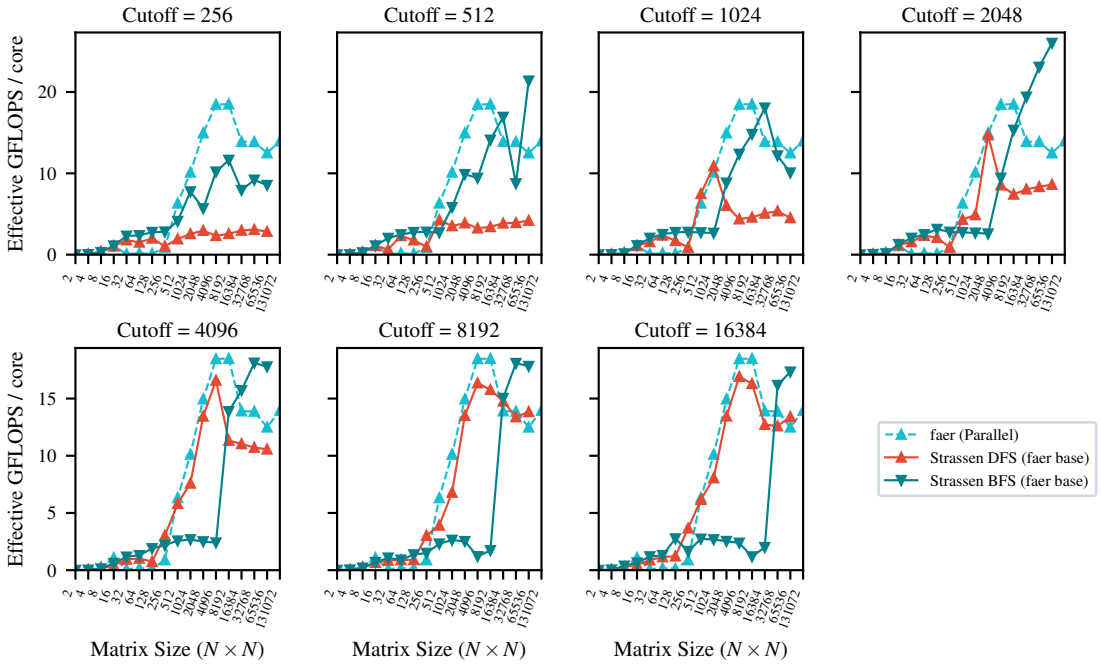


(b) Parallel faer base.

Figure 4: Parallel performance comparison (normalized per core) of Strassen variants (DFS, BFS, Hybrid) against parallel gemm, across cutoff sizes (first row), and recursion levels (second row).



(a) Parallel dgemm base.



(b) Parallel faer base.

Figure 5: Parallel matrix multiplication performance comparison (normalized per core) of Strassen variants (DFS, BFS, Hybrid) against parallel gemm across size cutoffs.

5.6 Numerical comparison with Benson&Ballard Strassen

We perform a numerical performance comparison between our optimized Rust implementation of Strassen's algorithm and the reference C++ implementation by Benson and Ballard [1]. The results are shown in Figure 6. We compare the algorithms across one, two, and three levels of recursion using different thread scheduling paradigms: Sequential, Parallel DFS, Parallel BFS, and Parallel Hybrid. We see that our algorithm to compete especially in the sequential case, where the dgemm variant keeps head. In DFS case, Ballard code outperforms our algorithm, we think this is due their better parallelization of additions.

6 Conclusion & Further work

- **Profiling and SIMD** We believe that code can be made more efficient by doing a more fine refactory with profiling result. Many algorithm steps can be optimizing by introducing more SIMD operations.
- **Gemm comparison:** Though we choose mkl-dgemm for a better comparison, our ffi bindings easily extend to any other gemm, such as OpenBlas, which we leave to further work.
- **Gpu/distributed version:** We would like to port the project to GPU or distributed computing to extend results with higher parallelism and test larger sizes.
- **Run on Xeon:** The project could be run on high performance Intel Xeon nodes on cluster, which might show impact of Intel MKL dgemm propetary optimizations.

7 Work reproducibility

All the code is available at the repository fast-matmul.

8 Acknowledgments

We acknowledge the authors of [2] for providing instructions for linking MKL with Rust, and the department of Mathematics of the University of Pisa for offering access to their HPC cluster.

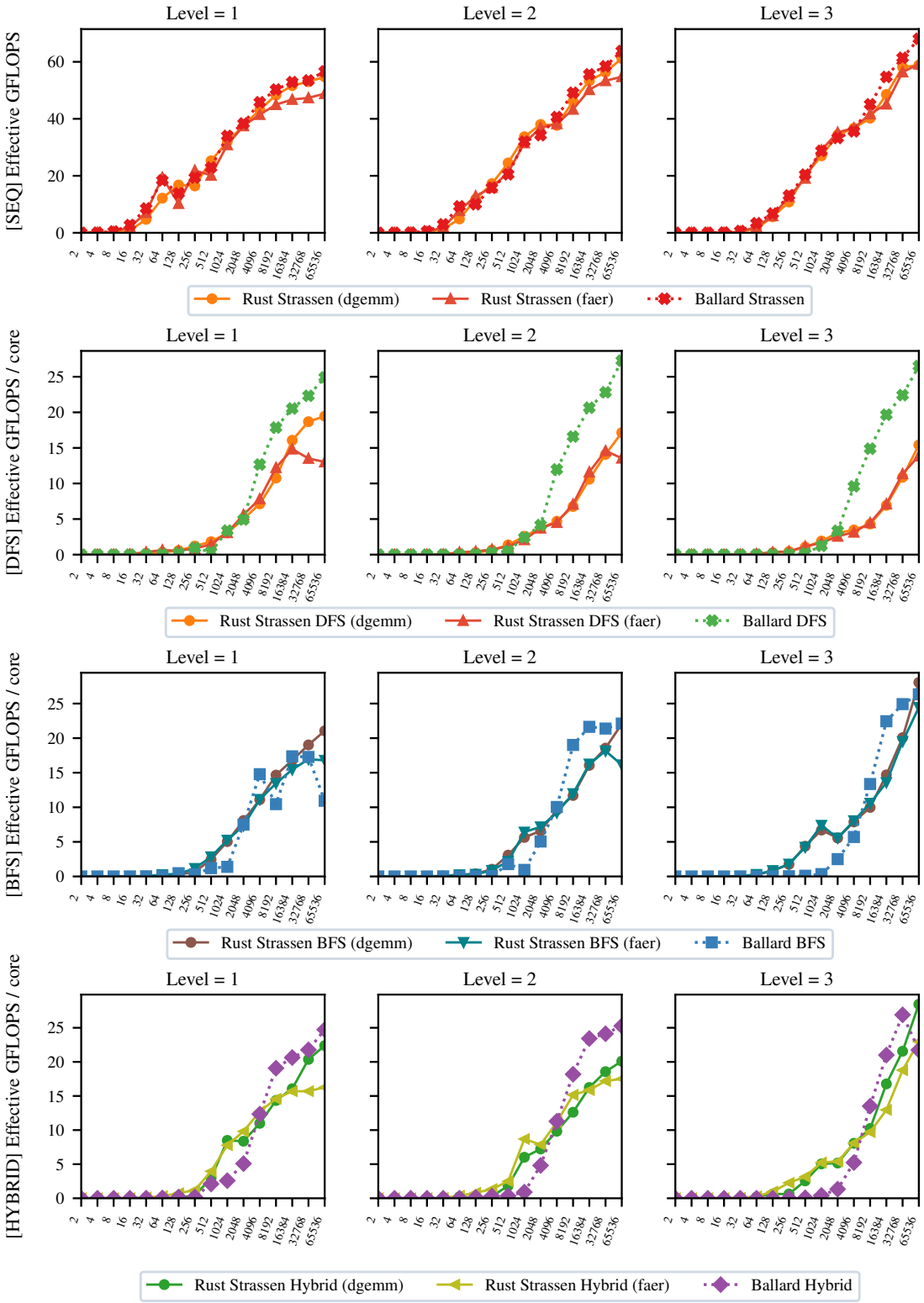


Figure 6: Performance comparison of our Rust Strassen implementation (using dgemm and faer base cases) against the Benson & Ballard reference benchmarks, sequential and parallel execution.

References

[1] Austin R. Benson and Grey Ballard. A framework for practical parallel fast matrix multiplication. In *Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP 2015, page 42–53, New York, NY, USA, 2015. Association for Computing Machinery.

[2] Luca Lombardo and Fabio Durastante. Evaluating rust for sparse matrix kernels in scientific computing, 2026.

[3] Grey Ballard and Tamara G. Kolda. *Tensor Decompositions for Data Science*. Cambridge University Press, 2025.

[4] S. Winograd. On multiplication of 2 × 2 matrices. *Linear Algebra and its Applications*, 4(4):381–388, 1971.

[5] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.

[6] Alexey V Smirnov. The bilinear complexity and practical algorithms for matrix multiplication. *Computational Mathematics and Mathematical Physics*, 53(12):1781–1795, 2013.

[7] Benjamin Lipshitz, Grey Ballard, James Demmel, and Oded Schwartz. Communication-avoiding parallel strassen: Implementation and performance. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pages 1–11, 2012.

[8] S. Huss-Lederman, E.M. Jacobson, J.R. Johnson, A. Tsao, and T. Turnbull. Implementation of strassen's algorithm for matrix multiplication. In *Supercomputing '96:Proceedings of the 1996 ACM/IEEE Conference on Supercomputing*, pages 32–32, 1996.